

Methods 3: Multilevel Statistical Modeling and Machine Learning

Week 47: *Neural Networks*

November 18, 2025

Job advertisement

Bliv mentor hos Rådgivnings- og støttecentret, AU

Er du imødekommende, struktureret og fleksibel - og har du lyst til at være mentor for en studerende? Så søg stillingen som mentor hos RSC!

Vi tilbyder:

- ◆ Et meningsfuldt job, fleksible timer og god løn
- ◆ 10-20 arbejdstimer pr. mentorforløb i semestret
- ◆ Individuel supervision og kompetenceudvikling

Typiske arbejdsopgaver:

- ◆ Faglig sparring
- ◆ Navigering i studiets krav og rammer
- ◆ At være lyttende og stille spørgsmål
- ◆ Strukturering og planlægning af studiehverdagen

Kender du nogen, der kunne være interesseret? Del gerne – vi søger aktivt, men uopfordrede ansøgninger er kun en dejlig overraskelse!

Rådgivnings- og støttecentret udbyder understøttelse til studerende med psykiske eller neurologiske funktionsnedsættelser

Lige nu søger vi mentorer fra:

Cognitive Science

Send ansøgning, CV og seneste karakterudskrift til mentor.rsc.udd@au.dk
[Emnefelt: "Cognitive Science_Ansøgning"]



Læs mere om at være mentor på vores hjemmeside ➡



The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

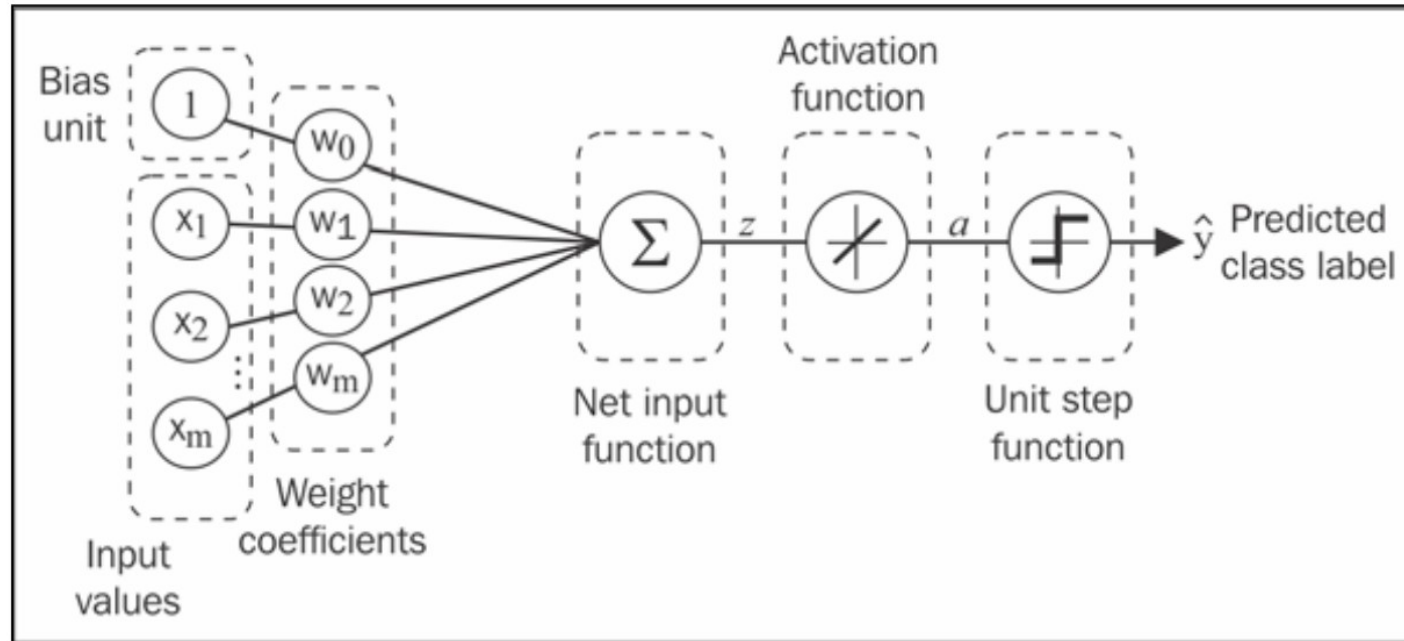
Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Recap – PCA

- PCA can be used to reduce complexity of data and thus the degree of overfitting
- *n_components* in PCA can be treated as a hyperparameter
- PCA can be seen as a way of transforming one's coordinate system and creating new basis vectors (axes)

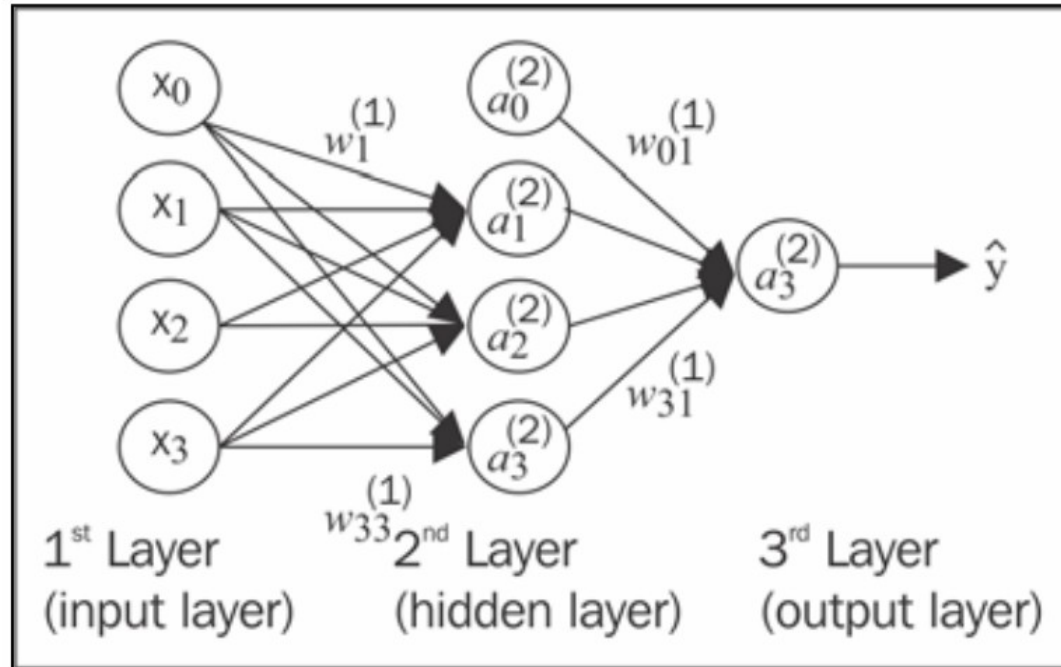
Recap – ADALINE

SINGLE LAYER NEURAL NETWORK



(p. 343: Raschka, 2015)

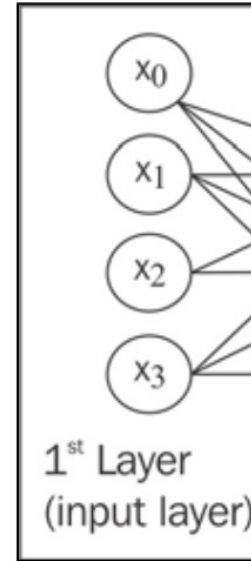
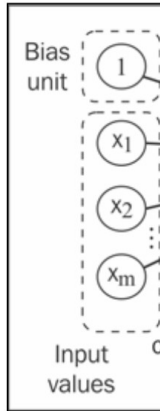
Multi-layer neural network



(p. 345: Raschka, 2015)

Multi-layer neural network

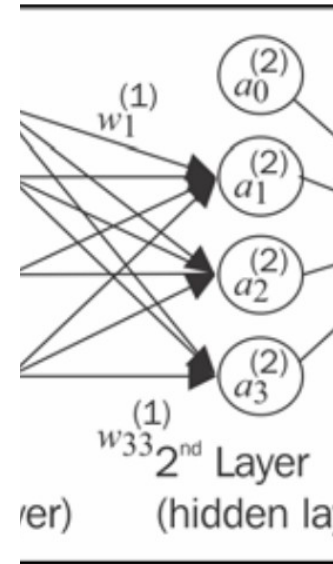
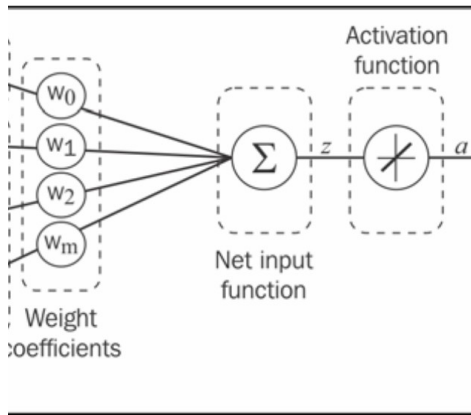
SIMILAR INPUT STRUCTURES



(p. 345: Raschka, 2015)

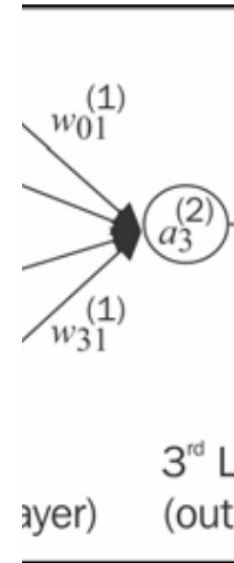
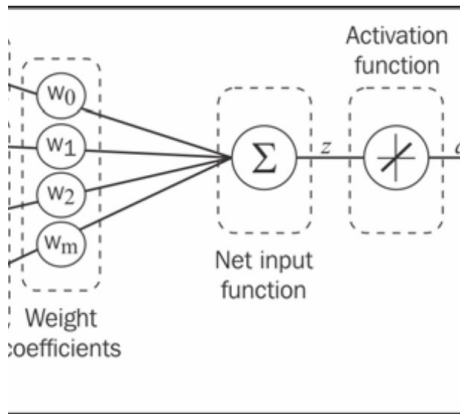
Multi-layer neural network

SIMILAR WEIGHT COEFFICIENTS AND ACTIVATION FUNCTIONS, BUT *MORE* OF THEM



(p. 345: Raschka, 2015)

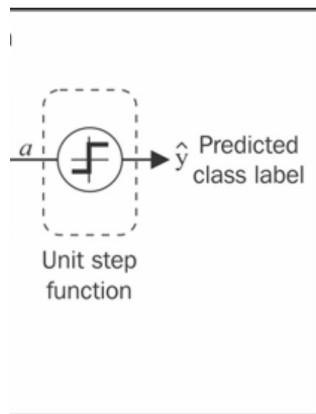
Multi-layer neural network AND REPEATED ...



(p. 345: Raschka, 2015)

Multi-layer neural network

OFTEN SOFT-MAX USED TO DECIDE



$$p_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

→ $\hat{y}$

layer
put layer

(p. 345: Raschka, 2015)

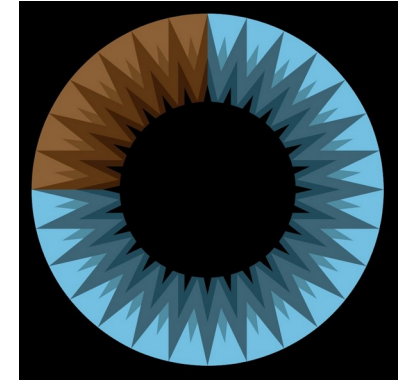
Rough definitions

- Single layer: no layer between input and output layers
- Multi-layer: a hidden layer between input and output layers
- Deep neural network (hidden layers > 1)

Learning goals and outline

Neural Networks

- 1) Understanding the idea of a multilayer network
- 2) Understanding that it is partly a string of operations that we already know how to perform, e.g. logistic regression or linear regression
- 3) Getting the beginning of an understanding of backpropagation and how it can improve gradient descent



[/home/lau/Videoklip/methods3/But what is a neural network?](/home/lau/Videoklip/methods3/But%20what%20is%20a%20neural%20network?) | Chapter 1, Deep learning |

But what is a neural network? | Chapter 1, Deep learning , 3Blue1Brown,
<https://www.youtube.com/watch?v=aircAruvnKk>

The input

$$X_{n \times m} = \begin{bmatrix} X_{11} & X_{12} & \dots & X_{1(m-1)} & X_{1m} \\ X_{21} & X_{22} & \dots & X_{2(m-1)} & X_{2m} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ X_{(n-1)1} & X_{(n-1)2} & \dots & X_{(n-1)(m-1)} & X_{(n-1)m} \\ X_{n1} & X_{n2} & \dots & X_{n(m-1)} & X_{nm} \end{bmatrix}$$

n : number of observations

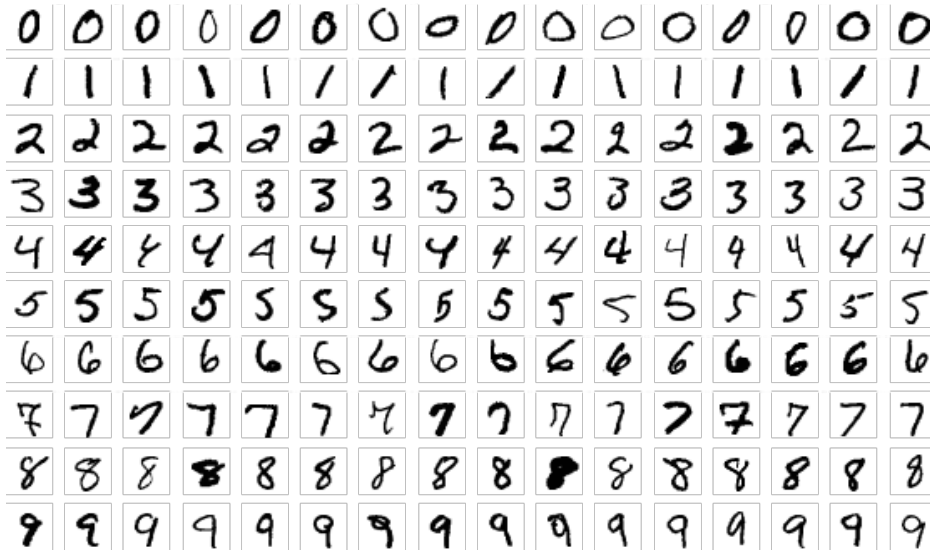
m : number of features (predictors)

The input

$$y_n = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_{n-1} \\ y_n \end{pmatrix}$$

n : number of observations

MNIST database



$Y_{10 \times 16}$

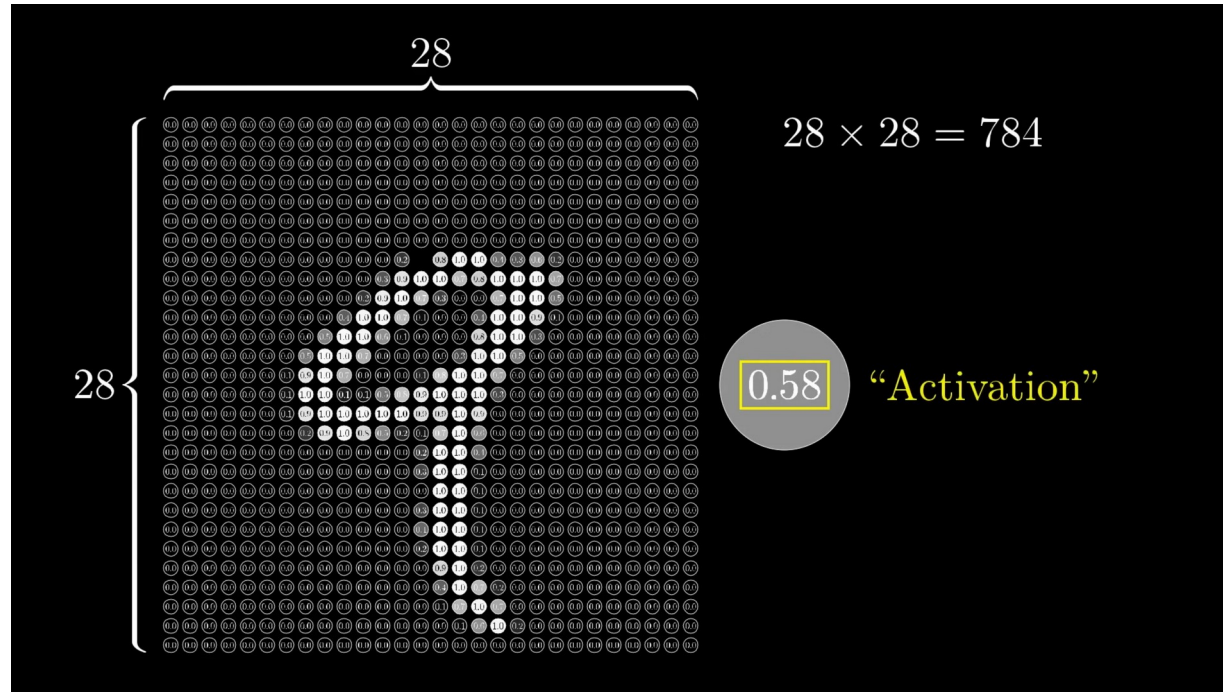
By Suvarjanprasai - Own work, CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=132282871>

The **MNIST database** (Modified *National Institute of Standards and Technology database*^[1]) is a large *database* of handwritten digits that is commonly used for *training* various *image processing* systems.^{[2][3]} The database is also widely used for training and testing in the field of *machine learning*.^{[4][5]} It was created by "re-mixing" the samples from NIST's original datasets.^[6] The creators felt that since NIST's training dataset was taken from American *Census Bureau* employees, while the testing dataset was taken from *American high school* students, it was not well-suited for machine learning experiments.^[7] Furthermore, the black and white images from NIST were *normalized* to fit into a 28x28 pixel bounding box and *anti-aliased*, which introduced grayscale levels.^[7]

$X_{160 \times 28 \times 28}$

https://en.wikipedia.org/wiki/MNIST_database

The input



3Blue1Brown: *But what is a neural network?* | Chapter 1, Deep learning
<https://www.youtube.com/watch?v=aircAruvnKk>

How do we reshape?

numpy.reshape

```
numpy.reshape(a, /, shape=None, *, newshape=None, order='C', copy=None)
```

Gives a new shape to an array without changing its data.

[\[source\]](#)

Parameters:

a : *array_like*

Array to be reshaped.

shape : *int or tuple of ints*

The new shape should be compatible with the original shape. If an integer, then the result will be a 1-D array of that length. One shape dimension can be -1. In this case, the value is inferred from the length of the array and remaining dimensions.

numpy.squeeze

```
numpy.squeeze(a, axis=None)
```

[\[source\]](#)

Remove axes of length one from *a*.

$y_{10 \times 16} \rightarrow y_{160}$

$X_{160 \times 28 \times 28} \rightarrow X_{160 \times 784}$

MNIST

- Original

- $X_{70000 \times 784}$
- y_{70000}

- Reduced

```
[9]: from sklearn.datasets import load_digits  
  
X, y = load_digits(return_X_y=True)  
  
print(X.shape)  
print(y.shape)  
  
(1797, 64)  
(1797,)
```

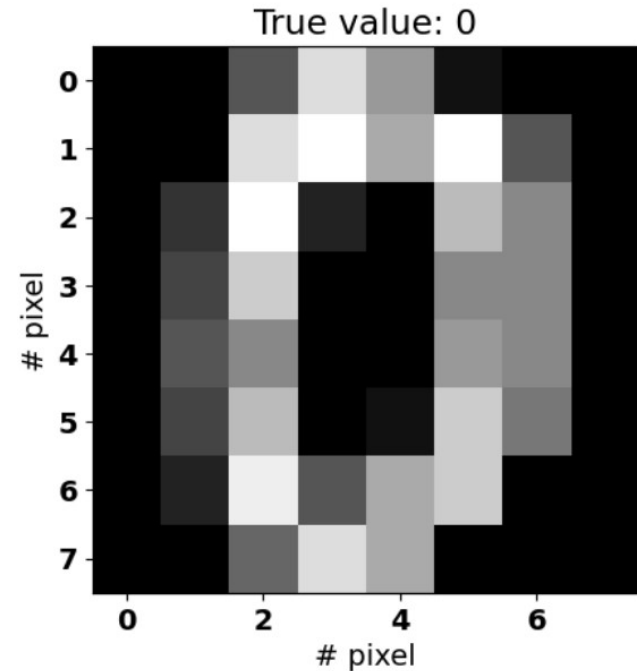
Reduced MNIST

```
### GET MNIST

from sklearn.datasets import load_digits

X, y = load_digits(return_X_y=True)

## plot first digit
first_image = np.reshape(X[0, :], (8, 8))
plt.figure()
plt.imshow(first_image, cmap='gray')
plt.xlabel('# pixel')
plt.ylabel('# pixel')
plt.title('True value: ' + str(y[0]))
plt.show()
```



```

class NeuralNetMLP(object):
    """ Feedforward neural network / Multi-layer perceptron classifier.

    Parameters
    -----
    n_output : int
        Number of output units, should be equal to the
        number of unique class labels.
    n_features : int
        Number of features (dimensions) in the target dataset.
        Should be equal to the number of columns in the X array.
    n_hidden : int (default: 30)
        Number of hidden units.
    l1 : float (default: 0.0)
        Lambda value for L1-regularization.
        No regularization if l1=0.0 (default)
    l2 : float (default: 0.0)
        Lambda value for L2-regularization.
        No regularization if l2=0.0 (default)
    epochs : int (default: 500)
        Number of passes over the training set.
    eta : float (default: 0.001)
        Learning rate.
    alpha : float (default: 0.0)
        Momentum constant. Factor multiplied with the
        gradient of the previous epoch t-1 to improve
        learning speed
         $w(t) := w(t) - (\text{grad}(t) + \alpha * \text{grad}(t-1))$ 
    decrease_const : float (default: 0.0)
        Decrease constant. Shrinks the learning rate
        after each epoch via  $\text{eta} / (1 + \text{epoch} * \text{decrease\_const})$ 
    shuffle : bool (default: True)
        Shuffles training data every epoch if True to prevent circles.
    minibatches : int (default: 1)
        Divides training data into k minibatches for efficiency.
        Normal gradient descent learning if k=1 (default).
    random_state : int (default: None)
        Set random state for shuffling and initializing the weights.

    Attributes
    -----
    cost_ : list
        Sum of squared errors after each epoch.

```

Raschka's network

NOTHING MYSTERIOUS

```

def fit(self, X, y, print_progress=False):
    """ Learn weights from training data.

    Parameters
    -----
    X : array, shape = [n_samples, n_features]
        Input layer with original features.
    y : array, shape = [n_samples]
        Target class labels.
    print_progress : bool (default: False)
        Prints progress as the number of epochs
        to stderr.

    Returns:
    -----
    self

    """

```

331 lines of code

Three steps

- 1) Starting at the input layer, we **forward propagate** the patterns of the training data through the network to generate an output
- 2) Based on the network's output, we calculate the error that we want to minimise using a **cost function**
- 3) We **backpropagate** the error, find its derivative with respect to each weight in the network, and update the model

(p. 347: Raschka, 2015)

Zeroth steps

ENCODING LABELS

```
self.cost_ = []
X_data, y_data = X.copy(), y.copy()
y_enc = self._encode_labels(y, self.n_output)

delta_w1_prev = np.zeros(self.w1.shape)
delta_w2_prev = np.zeros(self.w2.shape)

for i in range(self.epochs):

    # adaptive learning rate
    self.eta /= (1 + self.decrease_const*i)

    if print_progress:
        sys.stderr.write('\rEpoch: %d/%d' % (i+1, self.epochs))
        sys.stderr.flush()

    if self.shuffle:
        idx = np.random.permutation(y_data.shape[0])
        X_data, y_enc = X_data[idx], y_enc[:, idx]

    mini = np.array_split(range(y_data.shape[0]), self.minibatches)
    for idx in mini:
```

```
def _encode_labels(self, y, k):
    """Encode labels into one-hot representation

    Parameters
    -----
    y : array, shape = [n_samples]
        Target values.

    Returns
    -----
    onehot : array, shape = (n_labels, n_samples)

    """
    onehot = np.zeros((k, y.shape[0]))
    for idx, val in enumerate(y):
        onehot[val, idx] = 1.0
    return onehot
```

```
[7]: nn._encode_labels(y, 10)
```

```
[7]: array([[1., 0., 0., ..., 0., 0., 0.],
            [0., 1., 0., ..., 0., 0., 0.],
            [0., 0., 1., ..., 0., 0., 0.],
            ...,
            [0., 0., 0., ..., 0., 0., 0.],
            [0., 0., 0., ..., 1., 0., 1.],
            [0., 0., 0., ..., 0., 1., 0.]])
```

1) Starting at the input layer, we **forward propagate** the patterns of the training data through the network to generate an output

Feedforward

FIRST STEP

```
# feedforward
a1, z2, a2, z3, a3 = self._feedforward(X_data[idx],
                                       self.w1,
                                       self.w2)
```

```
def _initialize_weights(self):
    """Initialize weights with small random numbers."""
    w1 = np.random.uniform(-1.0, 1.0,
                           size=self.n_hidden*(self.n_features + 1))
    w1 = w1.reshape(self.n_hidden, self.n_features + 1)
    w2 = np.random.uniform(-1.0, 1.0,
                           size=self.n_output*(self.n_hidden + 1))
    w2 = w2.reshape(self.n_output, self.n_hidden + 1)
    return w1, w2
```

```
def _feedforward(self, X, w1, w2):
    """Compute feedforward step

    Parameters
    -----
    X : array, shape = [n_samples, n_features]
        Input layer with original features.
    w1 : array, shape = [n_hidden_units, n_features]
        Weight matrix for input layer -> hidden layer.
    w2 : array, shape = [n_output_units, n_hidden_units]
        Weight matrix for hidden layer -> output layer.

    Returns
    -----
    a1 : array, shape = [n_samples, n_features+1]
        Input values with bias unit.
    z2 : array, shape = [n_hidden, n_samples]
        Net input of hidden layer.
    a2 : array, shape = [n_hidden+1, n_samples]
        Activation of hidden layer.
    z3 : array, shape = [n_output_units, n_samples]
        Net input of output layer.
    a3 : array, shape = [n_output_units, n_samples]
        Activation of output layer.
```

Feedforward

w_1 & w_2 ; a_1 , a_2 & a_3 ; z_2 & z_3

```
a1 = self._add_bias_unit(X, how='column')
z2 = w1.dot(a1.T)
a2 = self._sigmoid(z2)
a2 = self._add_bias_unit(a2, how='row')
z3 = w2.dot(a2)
a3 = self._sigmoid(z3)
return a1, z2, a2, z3, a3
```

n : number of observations
 m : number of features
 h : number of hidden units
 t : number of output units

Discussion points:

What happens at each line here?

What should n_{output} be for the MNIST?

Given $\dim(X) = 160 \times 64$ and $n_{hidden}=50$
what are the dimensions of w_1 , w_2 , a_1 , a_2 ,
 a_3 , z_2 and z_3 ?

How should $\hat{y}$ be calculated?

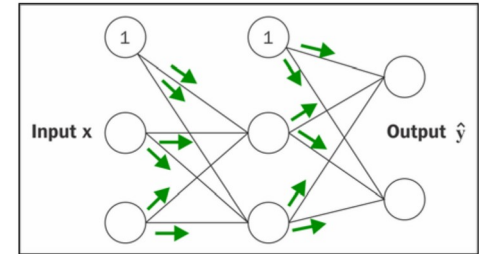
Feedforward steps summarised

$$\mathbf{Z}^{(2)} = \mathbf{W}^{(1)} \left[\mathbf{A}^{(1)} \right]^T \text{ (net input of the hidden layer)}$$

$$\mathbf{A}^{(2)} = \phi\left(\mathbf{Z}^{(2)}\right) \text{ (activation of the hidden layer)}$$

$$\mathbf{Z}^{(3)} = \mathbf{W}^{(2)} \mathbf{A}^{(2)} \text{ (net input of the output layer)}$$

$$\mathbf{A}^{(3)} = \phi\left(\mathbf{Z}^{(3)}\right) \text{ (activation of the output layer)}$$



2) Based on the network's output, we calculate the error that we want to minimise using a **cost function**

Cost function

LOGISTIC REGRESSION

```
def _get_cost(self, y_enc, output, w1, w2):
    """Compute cost function.

    Parameters
    -----
    y_enc : array, shape = (n_labels, n_samples)
        one-hot encoded class labels.
    output : array, shape = [n_output_units, n_samples]
        Activation of the output layer (feedforward)
    w1 : array, shape = [n_hidden_units, n_features]
        Weight matrix for input layer -> hidden layer.
    w2 : array, shape = [n_output_units, n_hidden_units]
        Weight matrix for hidden layer -> output layer.

    Returns
    -----
    cost : float
        Regularized cost.

    """
    term1 = -y_enc * (np.log(output))
    term2 = (1.0 - y_enc) * np.log(1.0 - output)
    cost = np.sum(term1 - term2)
    L1_term = self._L1_reg(self.l1, w1, w2)
    L2_term = self._L2_reg(self.l2, w1, w2)
    cost = cost + L1_term + L2_term
    return cost
```

```
def _L1_reg(self, lambda_, w1, w2):
    """Compute L1-regularization cost"""
    return (lambda_/2.0) * (np.abs(w1[:, 1:]).sum() +
                             np.abs(w2[:, 1:]).sum())
```

```
def _L2_reg(self, lambda_, w1, w2):
    """Compute L2-regularization cost"""
    return (lambda_/2.0) * (np.sum(w1[:, 1:] ** 2) +
                             np.sum(w2[:, 1:] ** 2))
```

$$l(\hat{\beta}|y) = \sum_{i=1}^n y_i \log(\hat{y}_i) + \log(1 - y_i)(1 - \hat{y}_i)$$

One vs rest is used

IN RASCHKA'S CODE

```
cost = self._get_cost(y_enc=y_enc[:, idx],  
                      output=a3,  
                      w1=self.w1,  
                      w2=self.w2)  
self.cost_.append(cost)
```



One-vs.-All (OvA), or sometimes also called **One-vs.-Rest (OvR)**, is a technique, us to extend a binary classifier to multi-class problems. Using OvA, we can train one classifier per class, where the particular class is treated as the positive class and the samples from all other classes are considered as the negative class. If we were to classify a new data sample, we would use our n classifiers, where n is the number of class labels, and assign the class label with the highest confidence to the particular sample. In the case of the perceptron, we would use OvA to choose the class label that is associated with the largest absolute net input value.

(p. 28: Raschka, 2015)

```
a1, z2, a2, z3, a3 = self._feedforward(X, self.w1, self.w2)  
y_pred = np.argmax(z3, axis=0)
```

$$l(\hat{\beta}|y) = \sum_{i=1}^n y_i \log(\hat{y}_i) + \log(1 - y_i)(1 - \hat{y}_i)$$

3) We **backpropagate** the error, find its derivative with respect to each weight in the network, and update the model

Interim summary

- So far, all we have done is applying tools we already know
- The main difference is that we apply it several times
 - We still calculate net input (z), also known as the linear predictor ($X\beta$)
 - We still apply an activation function $\varphi(z)$ to get data back on an interpretable scale
 - We still apply a quantiser function to make a decision ($\text{np.argmax}(z)$)

3) We **backpropagate** the error, find its derivative with respect to each weight in the network, and update the model

In the single layer case ...

WEIGHT UPDATING

$$\Delta w_j = -\eta \frac{\delta J}{\delta w_j} = \eta \sum_{i=1}^n \boxed{(\mathbf{y}_i - \phi(\mathbf{z}_i))} \boxed{x_j}$$

error

direction in
which to
update

Discussion point

Why does this become harder to do when hidden levels are introduced?

Cost function

WE NEED TO MINIMISE $J(\mathbf{w})$ FOR EACH WEIGHT, w

$$\frac{\partial}{\partial w_{j,i}^{(l)}} J(\mathbf{w}) = a_j^{(l)} \delta_i^{(l+1)}$$

error from next layer

activation
from current
layer

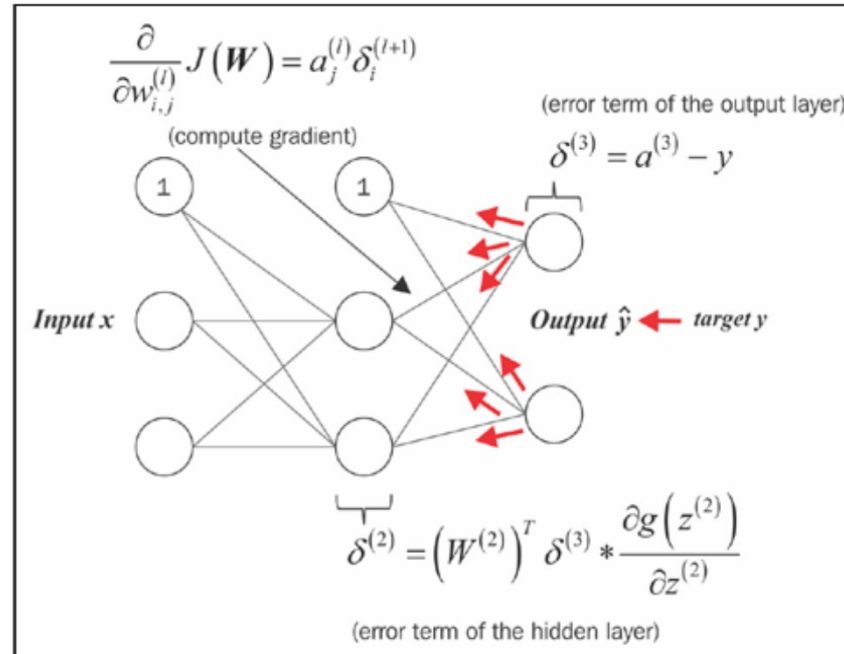
l : n_layer
 j : n_activation_unit
 i : n_sample

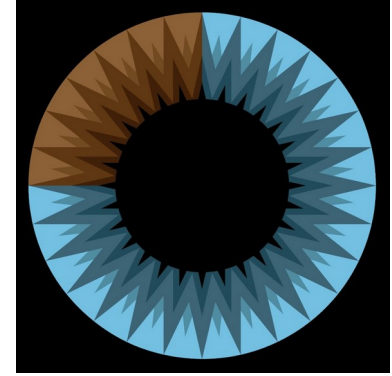
Intuitions

- When **error from next layer** is high, **activation from current layer** should be low.
- When **error from next layer is low**, **activation from current layer** can be high

$$\frac{\partial}{\partial w_{j,i}^{(l)}} J(\mathbf{w}) = a_j^{(l)} \delta_i^{(l+1)}$$

Backpropagation

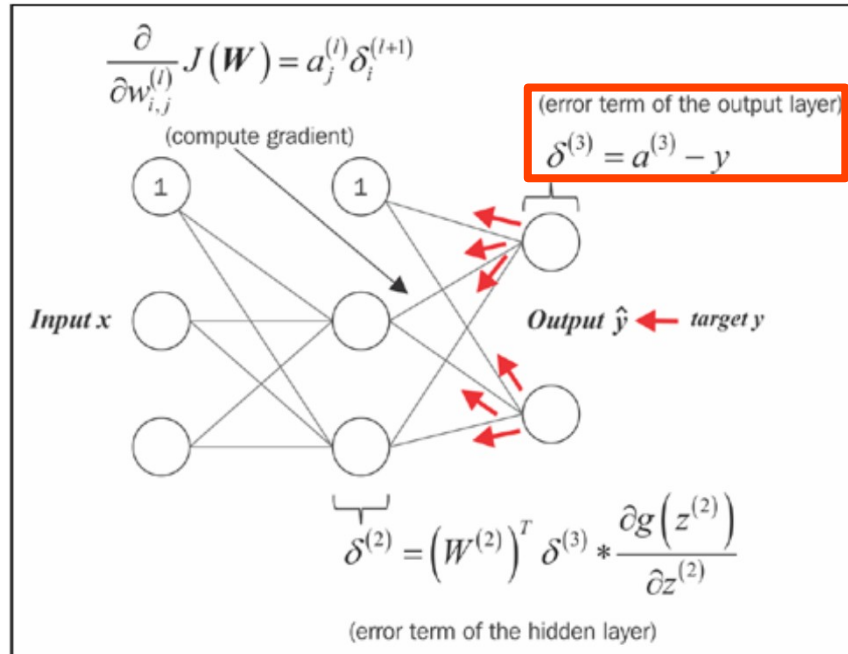




[/home/lau/Videoklip/methods3/What is backpropagation really doing? | Chapter 3, Deep learning \[Ilg3gGewQ5U\].mp4](#)

What is backpropagation really doing? | Chapter 3, Deep learning, 3Blue1Brown,
<https://www.youtube.com/watch?v=Ilg3gGewQ5U>

Backpropagation



```
# compute gradient via backpropagation
grad1, grad2 = self._get_gradient(a1=a1, a2=a2,
                                   a3=a3, z2=z2,
                                   y_enc=y_enc[:, idx],
                                   w1=self.w1,
                                   w2=self.w2)
```

```
# backpropagation
sigma3 = a3 - y_enc
z2 = self._add_bias_unit(z2, how='row')
sigma2 = w2.T.dot(sigma3) * self._sigmoid_gradient(z2)
sigma2 = sigma2[1:, :]
grad1 = sigma2.dot(a1)
grad2 = sigma3.dot(a2.T)
```

$$\delta_{t \times n}^{(3)}$$

Backpropagation

NOTE: CODE USES SIGMA INSTEAD OF DELTA

```
# backpropagation
sigma3 = a3 - y_enc
z2 = self._add_bias_unit(z2, how='row')
sigma2 = w2.T.dot(sigma3) * self._sigmoid_gradient(z2)
sigma2 = sigma2[1:, :]
grad1 = sigma2.dot(a1)
grad2 = sigma3.dot(a2.T)
```

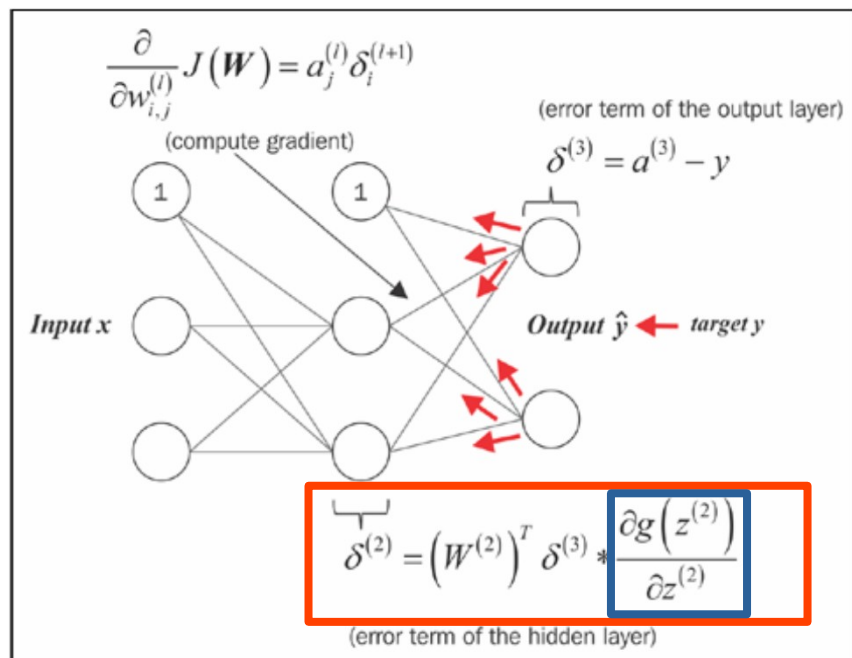
Discussion point

What does each of the four rows correspond to here?

0	1	2	3	4	5	6	7	8	9
0.90	0.05	0.00	0.00	0.05	0.00	0.00	0.00	0.00	0.00
1.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
-0.1	0.05	0.00	0.00	0.05	0.00	0.00	0.00	0.00	0.00

$\delta_{h \times n}^{(2)}$

Backpropagation



```
# backpropagation
sigma3 = a3 - y enc
z2 = self._add_bias_unit(z2, how='row')
sigma2 = w2.T.dot(sigma3) * self._sigmoid_gradient(z2)
sigma2 = sigma2[1:, :]
grad1 = sigma2.dot(a1)
grad2 = sigma3.dot(a2.T)
```

```
def _sigmoid_gradient(self, z):
    """Compute gradient of the logistic function"""
    sg = self._sigmoid(z)
    return sg * (1.0 - sg)
```

$$(a^{(2)} * (1 - a^{(2)}))$$

to which degree error is introduced at hidden layer

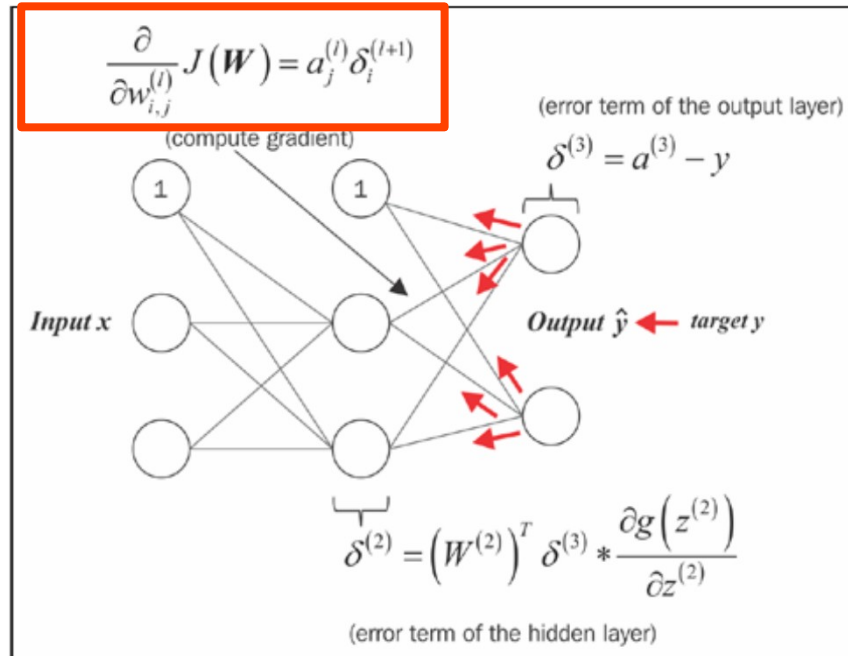
$$\delta^{(2)} = \mathbf{W}^{(2)T} \delta^{(3)} * \frac{\delta \phi(\mathbf{z}^{(2)})}{\delta \mathbf{z}^{(2)}}$$

error hidden layer error output layer direction in which to update

$$\frac{\delta \phi(\mathbf{z}^{(2)})}{\delta \mathbf{z}^{(2)}} = (\mathbf{a}^{(2)} * (1 - \mathbf{a}^{(2)}))$$

$$\delta^{(2)} = \mathbf{W}^{(2)T} \delta^{(3)} * (\mathbf{a}^{(2)} * (1 - \mathbf{a}^{(2)}))$$

h x n h x t t x n t x n t x n



$$\frac{\delta}{\delta w_{j,i}^{(l)}} J(W) = a_j^{(l)} \delta_i^{(l+1)}$$

Gradient calculation

```
def _get_gradient(self, a1, a2, a3, z2, y_enc, w1, w2):
    """ Compute gradient step using backpropagation.

    Parameters
    -----
    a1 : array, shape = [n_samples, n_features+1]
        Input values with bias unit.
    a2 : array, shape = [n_hidden+1, n_samples]
        Activation of hidden layer.
    a3 : array, shape = [n_output_units, n_samples]
        Activation of output layer.
    z2 : array, shape = [n_hidden, n_samples]
        Net input of hidden layer.
    y_enc : array, shape = (n_labels, n_samples)
        one-hot encoded class labels.
    w1 : array, shape = [n_hidden_units, n_features]
        Weight matrix for input layer -> hidden layer.
    w2 : array, shape = [n_output_units, n_hidden_units]
        Weight matrix for hidden layer -> output layer.

    Returns
    -----
    grad1 : array, shape = [n_hidden_units, n_features]
        Gradient of the weight matrix w1.
    grad2 : array, shape = [n_output_units, n_hidden_units]
        Gradient of the weight matrix w2.
```

```
# backpropagation
sigma3 = a3 - y_enc
z2 = self._add_bias_unit(z2, how='row')
sigma2 = w2.T.dot(sigma3) * self._sigmoid_gradient(z2)
sigma2 = sigma2[1:, :]
grad1 = sigma2.dot(a1)
grad2 = sigma3.dot(a2.T)
```

$$\frac{\delta}{\delta w_{j,i}^{(l)}} J(\mathbf{w}) = a_j^{(l)} \delta_i^{(l+1)}$$

```
# compute gradient via backpropagation
grad1, grad2 = self._get_gradient(a1=a1, a2=a2,
                                   a3=a3, z2=z2,
                                   y_enc=y_enc[:, idx],
                                   w1=self.w1,
                                   w2=self.w2)

delta_w1, delta_w2 = self.eta * grad1, self.eta * grad2
self.w1 -= (delta_w1 + (self.alpha * delta_w1_prev))
self.w2 -= (delta_w2 + (self.alpha * delta_w2_prev))
delta_w1_prev, delta_w2_prev = delta_w1, delta_w2
```

Summary

- Backpropagation allows for efficiently calculating the gradient
 - This means that the networks converge much more quickly than if calculating the gradient only using forward propagation
- Neural networks combine many pieces we have seen so far
 - Linear and logistic regression
 - Regularisation
 - Gradient descent

Learning goals and outline

Neural Networks

- 1) Understanding the idea of a multilayer network
- 2) Understanding that it is partly a string of operations that we already know how to perform, e.g. logistic regression or linear regression
- 3) Getting the beginning of an understanding of backpropagation and how it can improve gradient descent

The course plan

Week 37:

Lesson 0: What is it all about?

Class 0: Setting up R and Python — and recollection of the general linear model (Lau)

Week 38:

Lesson 1: Multilevel linear regression

Class 1: Modelling subject level effects – and how do they differ from group level effects? (Lydia)

Week 39:

Lesson 2: Beyond linear regression — link functions

Class 2: What to do when the response variable is not continuous? (Lau)

Week 40:

Lesson 3: Evaluating generalised linear mixed models — and the spectre of collinearity

Class 3: Code review (Lydia)

Week 41:

Lesson 4: Explanation versus prediction

Class 4: Summarising mixed models (Lydia)

Week 42:

no teaching (go fetch potatoes instead)

Week 43:

Lesson 5: Mid-way evaluation and machine learning introduction

Class 5: Getting Python running (Lau)

Week 44:

Lesson 6: Linear and logistic regression revisited (machine learning)

Class 6: Categorizing responses based on informed guesses (Lau)

Week 45:

Lesson 7: Model evaluation and hyperparameter tuning

Class 7: Code review (Lydia)

Week 46:

Lesson 8: Dimensionality reduction — principled component analysis (PCA)

Class 8: What to do with very rich data? (Lau)

Week 47:

Lesson 9: Neural networks

Class 9: Code review (Lau)

Week 48:

Lesson 10: Unsupervised classification

Class 10: Create video explainers of central concepts (shared resource) (Lydia)

Week 49:

Lesson 0 again: What was it all about? and final evaluation

Class 11: Summarising machine learning (Lydia)

Week 50:

Student presentations:

Class 12: Work on portfolios: Ask anything! (Lydia & Lau)

Next time

- Unsupervised learning
 - Without having the true labels, can we nonetheless find patterns in our datasets
- What is similarity?
 - That is, what kind of shapes do we allow for?

Reading questions

- What is a centroid?
- How do you pick k in k -means?
- What are *core*, *border* and *noise* points in DBSCAN?